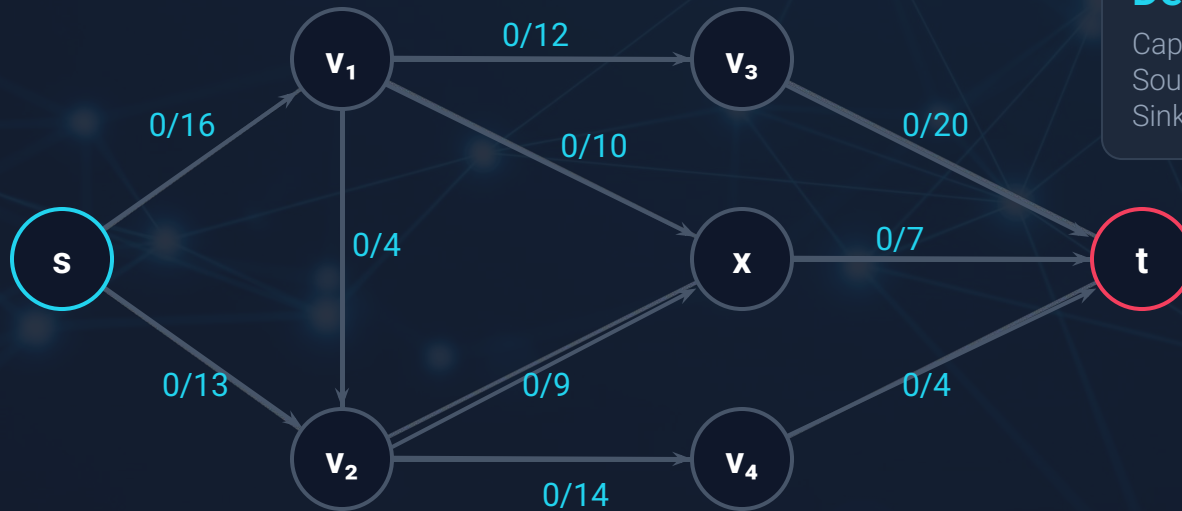


Maximum Flow and the Ford-Fulkerson Method

Luke Kaiser

Algorithm Analysis & Design

What is a Flow Network?

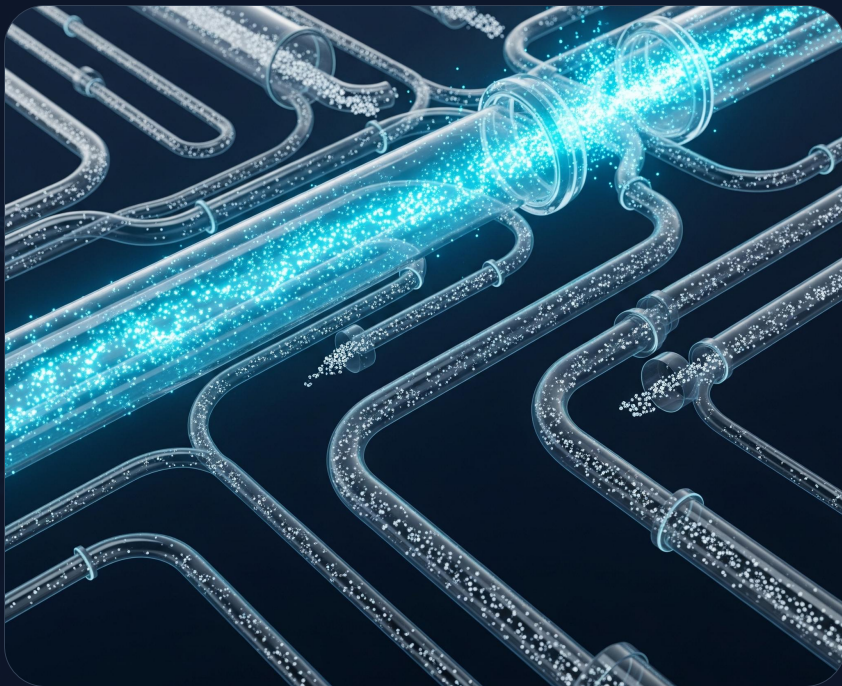


Definitions

Capacity: $c(u,v) \geq 0$
Source (s): Producer
Sink (t): Consumer

A directed graph $G = (V, E)$ where each edge has a stated maximum flow rate.

The Two Fundamental Rules of Flow



1. Capacity Constraint

The flow cannot be negative and must not exceed the conduit's stated capacity.

$$0 \leq f(u, v) \leq c(u, v)$$

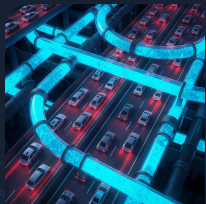
2. Flow Conservation

Total flow entering a vertex must equal the total flow leaving it (excluding s and t).

$$\sum_{v \in V} f(v, u) = \sum_{v \in V} f(u, v)$$

Goal: Find the maximum value flow from source (s) to sink (t) without violating these constraints.

Applications of Maximum Flow



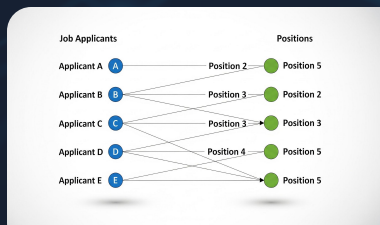
Physical Networks

Modeling liquids in pipes, traffic flow in road networks, and logistics in assembly lines.



Computer Networks

Determining maximum data throughput between communication nodes.



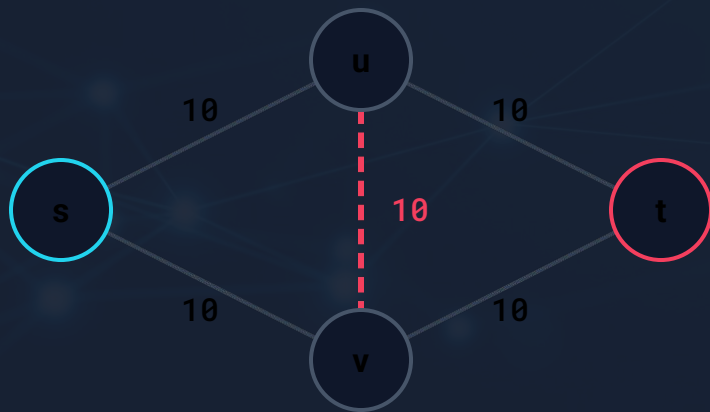
Maximum Bipartite Matching

Optimal pairing of sets, such as matching job applicants to positions or students to classes.

Uses 'supersource' and 'supersink' nodes to reduce complex pairing into a max-flow problem.

Maximum flow algorithms are versatile tools for both physical and abstract allocation problems.

The Greedy Problem



Why Greedy Fails

A purely greedy approach makes locally optimal choices that can block better global solutions.

- If we push 10 units through $s \rightarrow u \rightarrow v \rightarrow t$...
- We saturate the middle edge (u, v) .
- This greedy choice prevents us from using the full 20-unit capacity of the top and bottom paths.

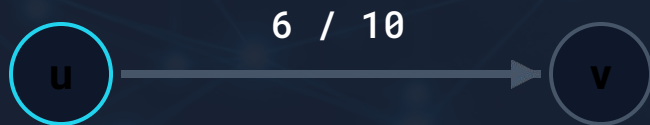
The Solution: "Undo" Ability

Algorithms need a way to reroute flow, which is why the **Residual Network** is essential.

A 4-node diamond graph demonstrates how a bad initial path choice can limit total throughput.

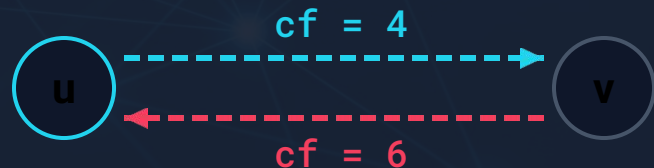
The Residual Network (G_f)

Original Flow Network (G)



Edge (u, v) has capacity $c=10$ and current flow $f=6$.

Residual Network (G_f)



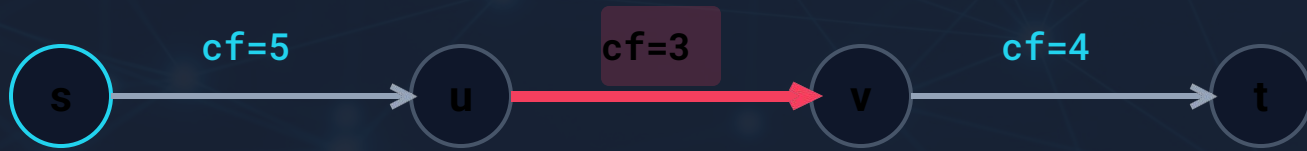
Forward: $c - f = 10 - 6 = 4$

Backward: $f = 6$ (cancellation)

Augmenting Paths & Bottlenecks

Defining the Bottleneck

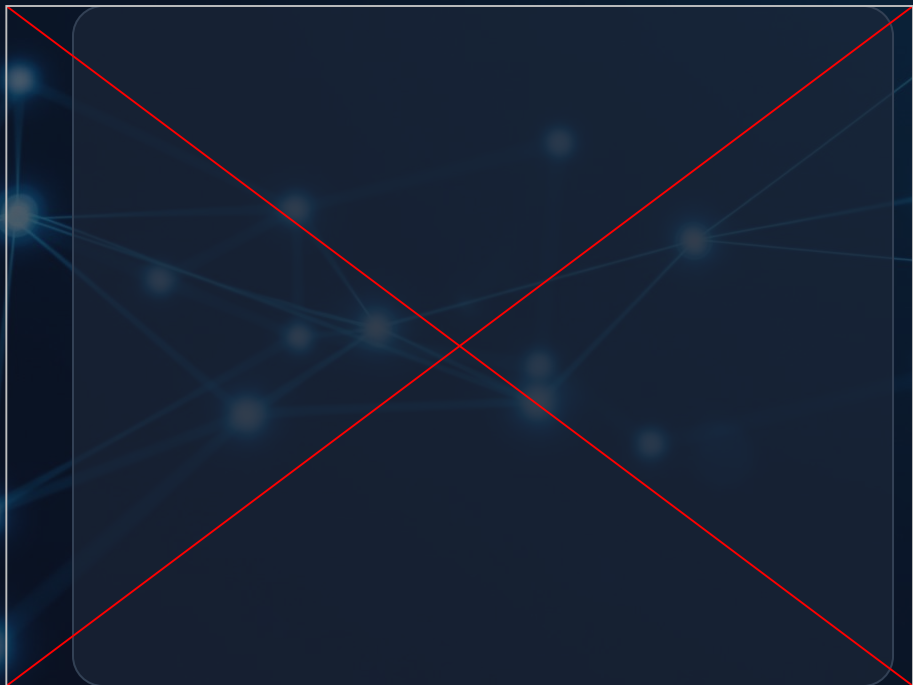
The residual capacity of an augmenting path p is limited by its weakest link which is the edge with the minimum residual capacity. This minimum value determines how much flow can be pushed across the entire path.



$$cf(p) = \min(5, 3, 4) \\ = 3$$

Pushing flow along this path increases total flow $|f|$ by exactly $cf(p)$.

The Ford-Fulkerson "Method"



High-Level Pseudocode

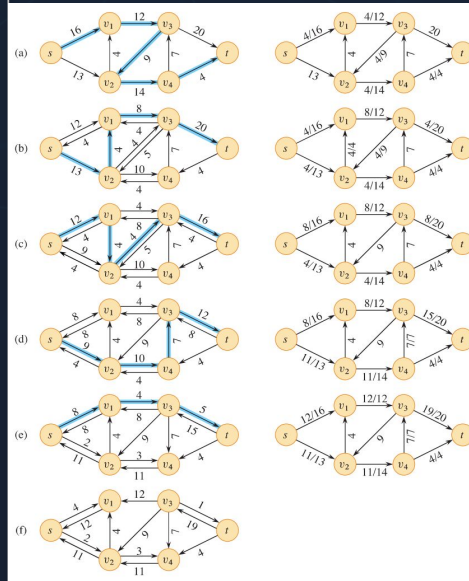
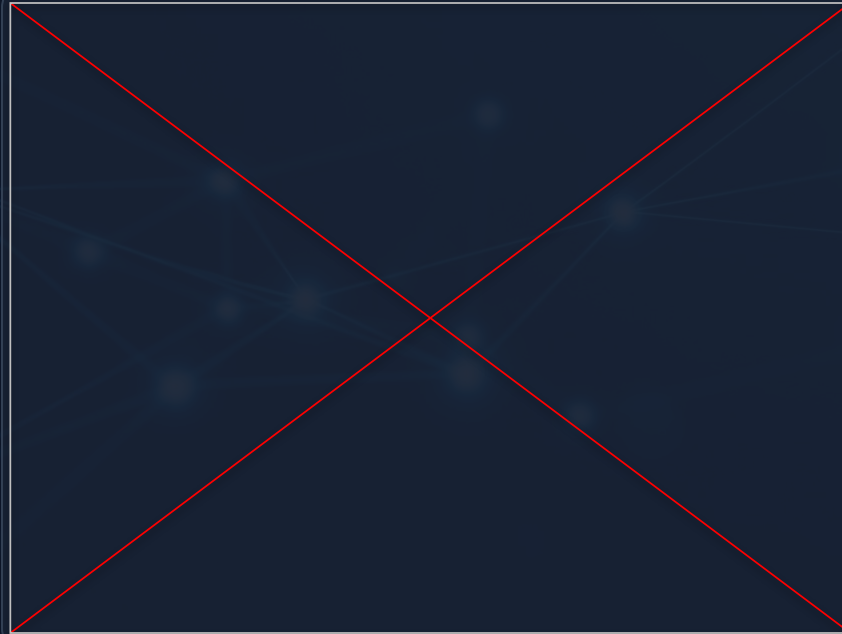
1. Initialize flow f to 0
2. **While** path p exists in G_f :
 Augment f along p
3. Return f

The Method terminology reflects flexibility in implementation: **how you find the path** determines the complexity.

Core logic: Iteratively push flow until the **Residual Network** has no more **Augmenting Paths**.

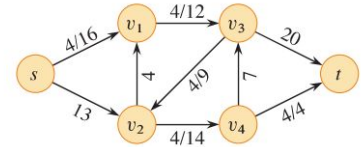
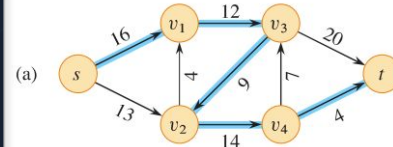
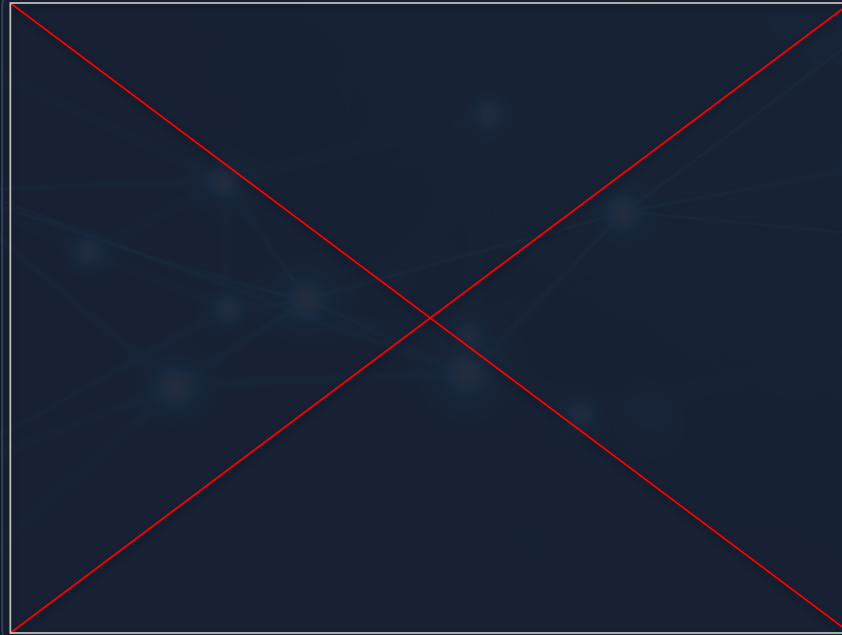
Ford-Fulkerson is the foundational approach for solving maximum flow problems across diverse network types.

Step-by-Step Example introduction



- **Initial State:** All pipes are completely empty (Flow = 0).
- **The Goal:** Push as much total flow as physically possible from the Source (s) to the Sink (t).
- **The Method:** We will iteratively find "augmenting paths" (routes with available space) and push flow until every possible route is blocked.

Step-by-Step Example (Iteration 1)

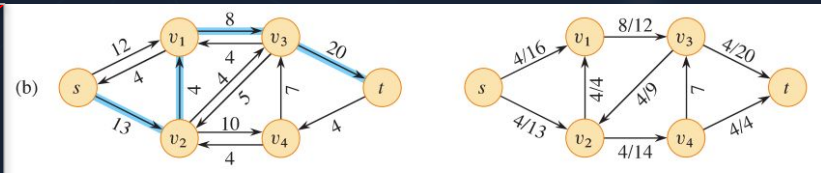
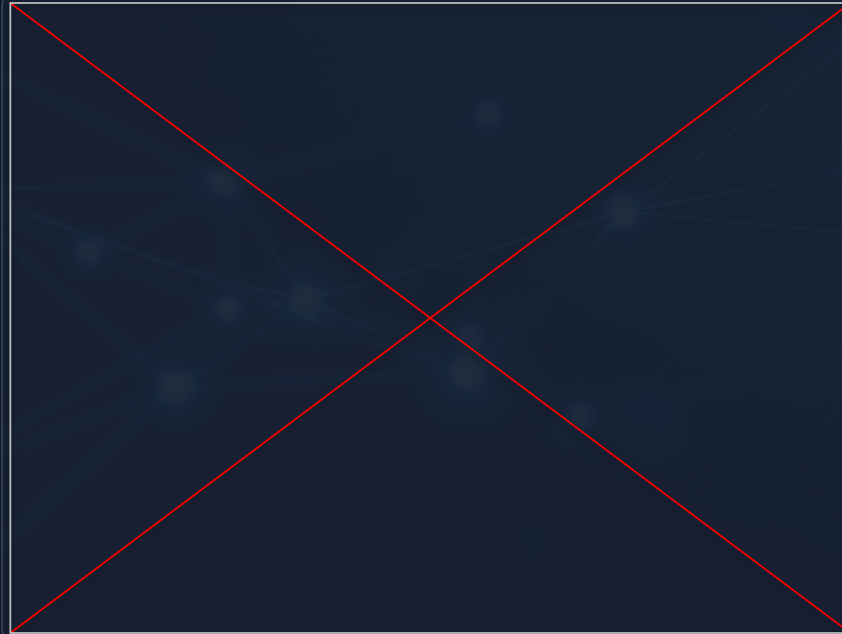


Path Found: $s \rightarrow v1 \rightarrow v3 \rightarrow v2 \rightarrow v4 \rightarrow t$

Bottleneck: 4 units (limited by the $v4 \rightarrow t$ edge)

Action: Push 4 units of flow along the entire path.

Step-by-Step Example (Iteration 2)



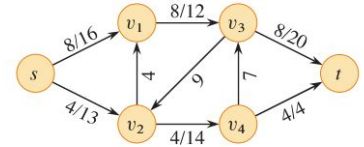
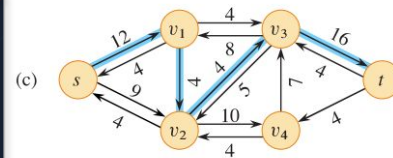
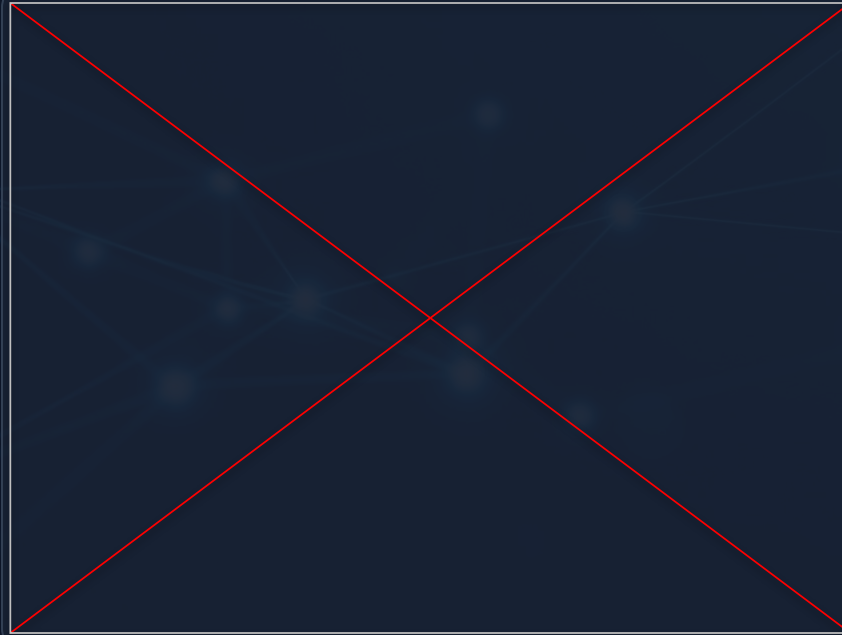
Path Found: $s \rightarrow v_2 \rightarrow v_1 \rightarrow v_3 \rightarrow t$

Bottleneck: 4 units (limited by the $v_2 \rightarrow v_1$ edge)

Action: Push 4 units of flow along this new path.

Note: The $v_1 \rightarrow v_3$ edge now carries 8 total units.

Step-by-Step Example (Iteration 3)



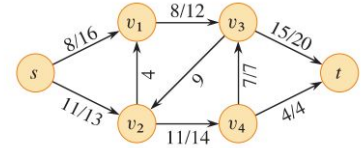
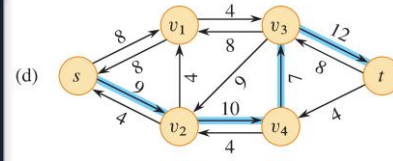
Path Found: $s \rightarrow v1 \rightarrow v2 \rightarrow v3 \rightarrow t$

Bottleneck: 4 units

Using Residual Network: We route flow against the current of $v2 \rightarrow v1$ and $v3 \rightarrow v2$.

Action: This acts as an "undo" button. The flow on those two pipes drops to 0, freeing them up!

Step-by-Step Example (Iteration 4)

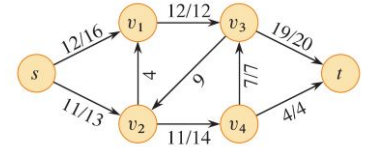
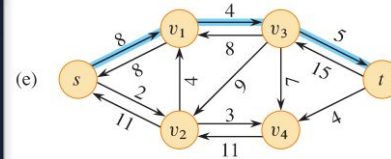
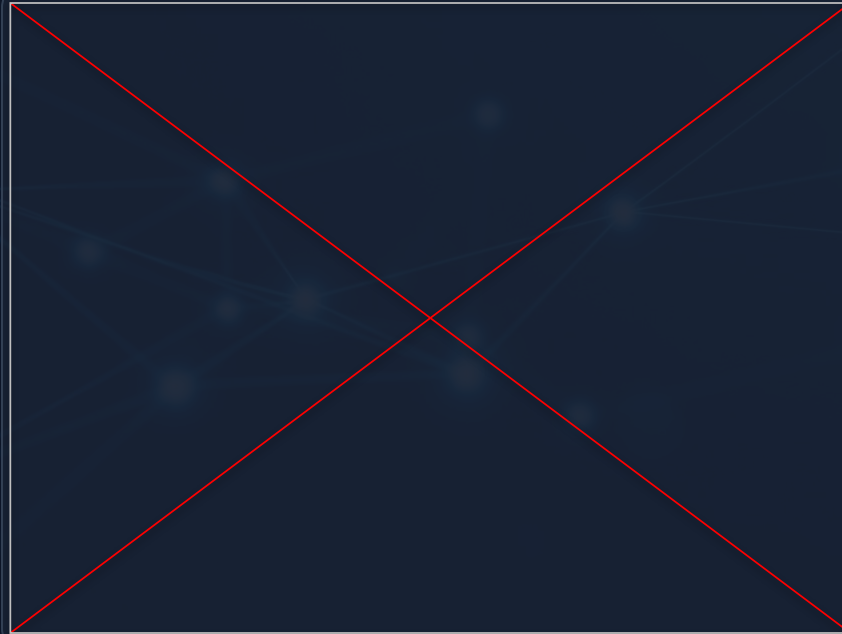


Path Found: $s \rightarrow v2 \rightarrow v4 \rightarrow v3 \rightarrow t$

Bottleneck: 7 units (limited by the $v4 \rightarrow v3$ edge)

Action: Push 7 units of flow along this bottom route.

Step-by-Step Example (Iteration 5)

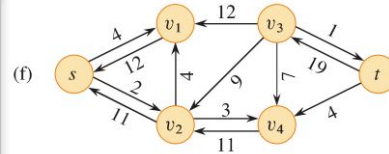


Path Found: $s \rightarrow v_1 \rightarrow v_3 \rightarrow t$

Bottleneck: 4 units (limited by the remaining space on $s \rightarrow v_1$ and $v_3 \rightarrow t$)

Action: Push the final 4 units of flow through the top route.

Step-by-Step Example End



Termination: No more unbroken paths with available capacity exist from Source to Sink.

Final Calculation: Add the total flow leaving the Source.

$$s \rightarrow v1 = 12$$

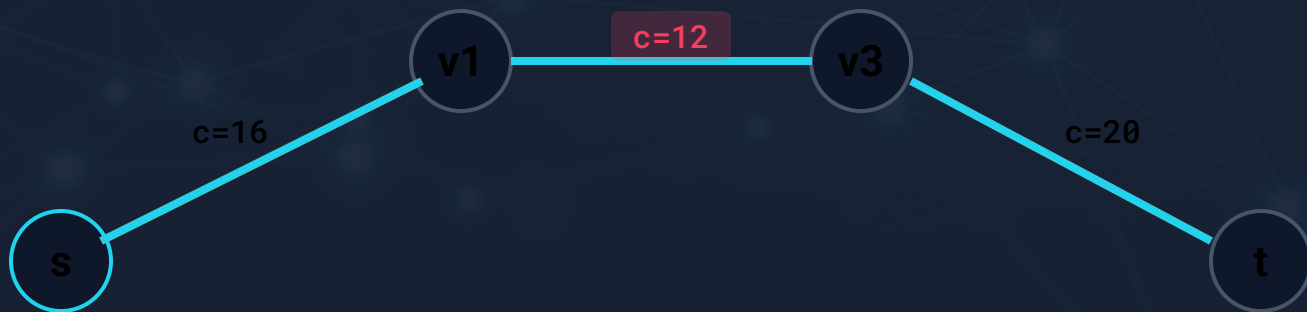
$$s \rightarrow v2 = 11$$

$$\text{Maximum Flow} = 23$$

Step-by-Step Example (Iteration 1)

Initial Network & First Path

Starting with zero flow, we identify the path $s \rightarrow v1 \rightarrow v3 \rightarrow t$. The bottleneck is determined by the edge with the minimum residual capacity ($v1, v3$).



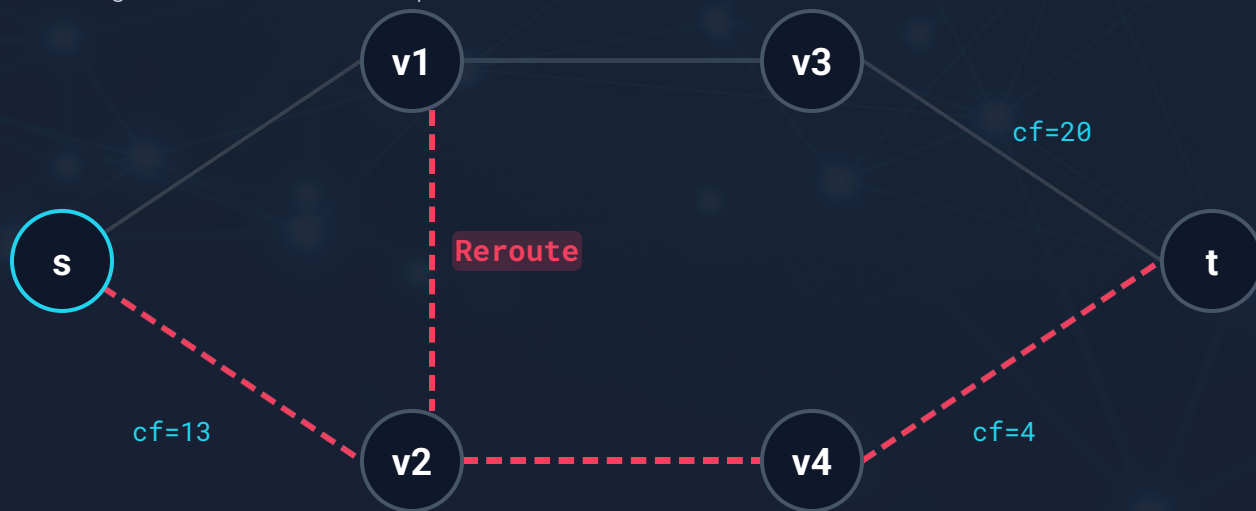
Bottleneck = 12

Total flow increased from 0 to 12.

Step-by-Step Example (Iterations 2 & 3)

Residual Network Updates

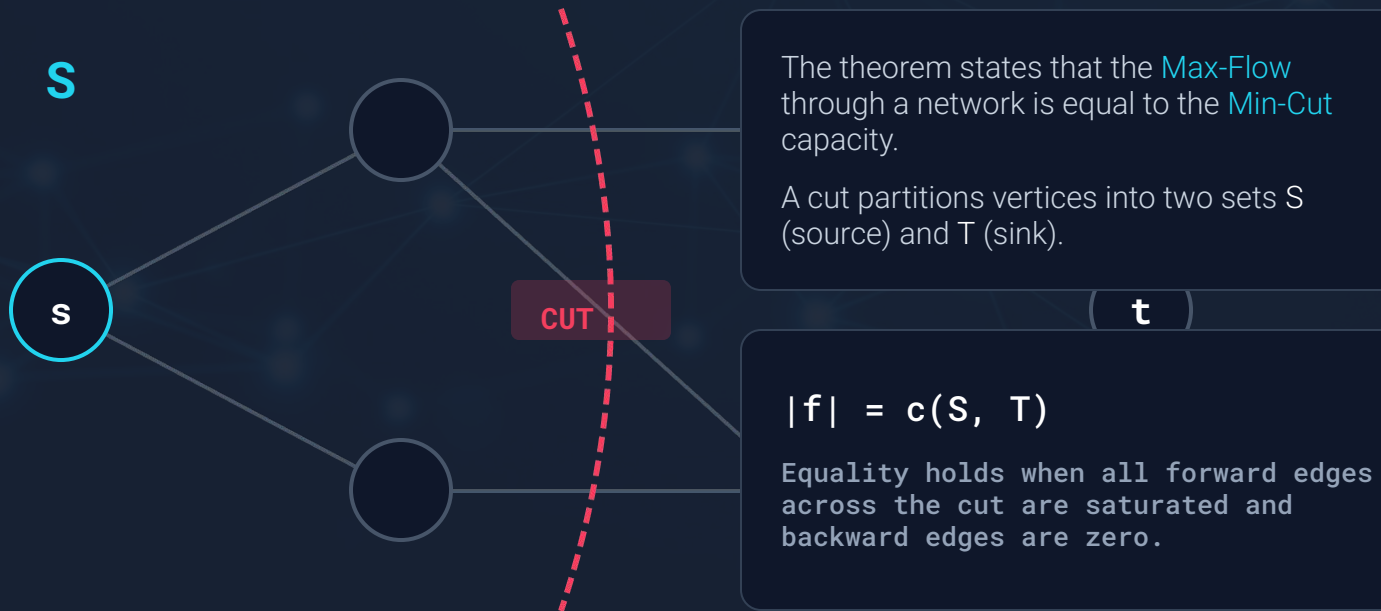
Iteration 2 adds flow along $s \rightarrow v2 \rightarrow v4 \rightarrow t$ (Bottleneck 4). Iteration 3 demonstrates the "cancellation" effect by using a backward edge to reroute flow and optimize the network.



Current Path: $s \rightarrow v2 \rightarrow v4 \rightarrow t$ (Iteration 2). Next: $v2 \rightarrow v1$ cancellation (Iteration 3).

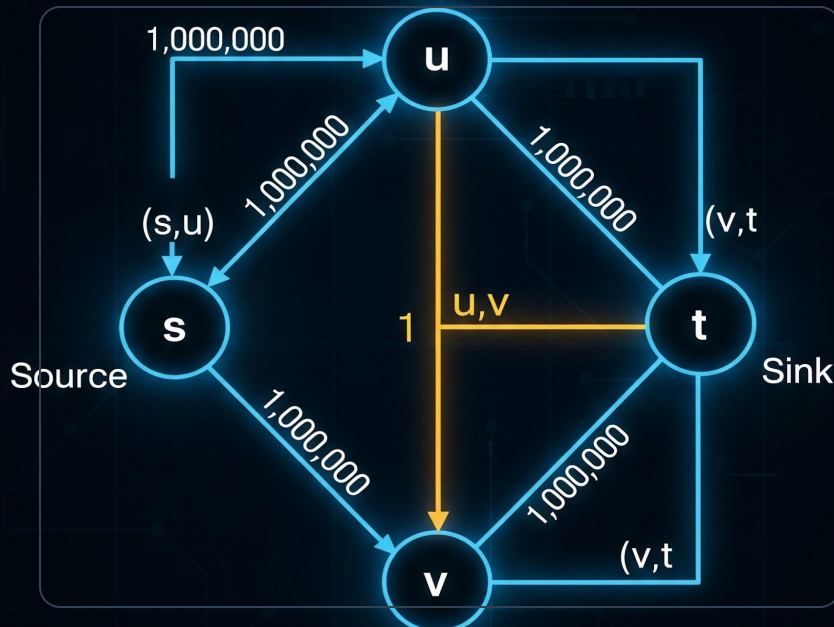
Total Flow = 16

Max-Flow Min-Cut Theorem



Termination criteria: The algorithm ends when no augmenting paths exist in G_f , indicating we've found the minimum cut.

The Bad Case & Edmonds-Karp



Pathological Scenario

Alternating paths through the middle edge increase flow by only 1 unit/iteration.

Iterations: 2,000,000

Edmonds-Karp Fix

Mandates **Breadth-First Search (BFS)** to find the shortest path. Bounds complexity to:

$$O(VE^2)$$

Complexity Analysis

Time Complexity

$$O(E |f^*|)$$

Each augmentation increases flow by ≥ 1 unit. With BFS/DFS taking $O(E)$, total time scales with max flow $|f^*|$.

Space Complexity

$$O(V + E)$$

Storage for both the original flow network and the residual graph using adjacency list representations.

V = Vertices, E = Edges, $|f^*|$ = Maximum Flow Value

Summary & Conclusions

Maximum Flow Problem

Maximize throughput from source to sink while strictly adhering to capacity and conservation constraints.

Ford-Fulkerson Method

Iteratively pushes flow along augmenting paths, using backward edges for rerouting and cancellation.

Key Insight

The algorithm terminates when no more augmenting paths exist, achieving $\text{Max-Flow} = \text{Min-Cut}$.